Day 86: Word2Vec (Skip-Gram) From Scratch, on a Real Small Corpus

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 3 | Complete study guide

Watch: Stanford CS224N, Lecture 1: Intro & Word Vectors (Stanford Online) | Vectoring Words (Word Embeddings) (Computerphile)

Script: `day86_word2vec_skipgram.py` · every number in this guide is copied by program from `day86_run_output.txt`, the real output of that script.

Day 85's word table learned from part-of-speech labels. Word2Vec learns one from raw text: each word is trained to predict the words around it. On the first 2,000,000 words of Wikipedia (text8), with nothing labelled, months end up next to months and numbers next to numbers: 0.970 of 90 listed words have a same-group nearest neighbour (mean of three seeds), against 0.156 for an untrained table. Analogies such as paris:france :: rome:? mostly fail at this size (0.140 and 0.263 right). Negative sampling makes it affordable: one batch is 282 times faster than with a full softmax.

0. Where today fits

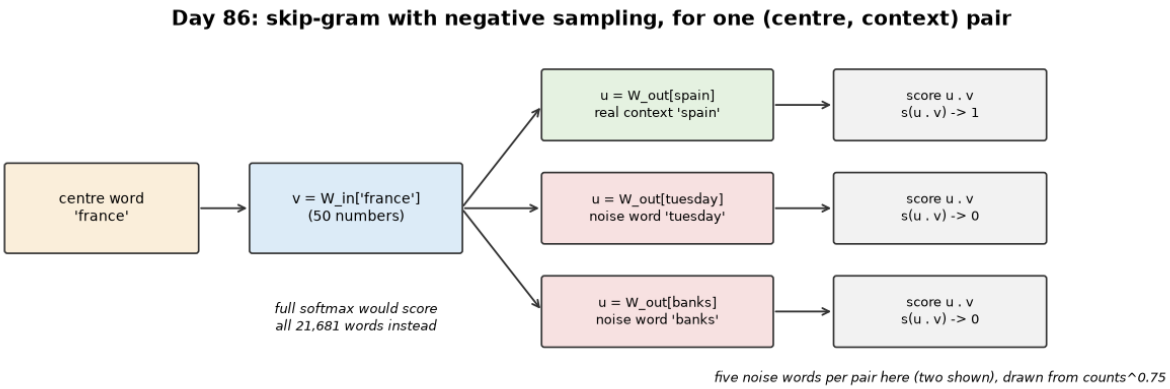
DEFINITION — start here, no prior knowledge assumed

A **word embedding** is a list of numbers for each word (here 50), learned so that words used in similar places get similar lists. Word2Vec's **skip-gram** model learns them from raw text: every word in turn is the **centre**, and the model learns to predict its **context**, the words within a few places of it. Words with similar contexts ('january', 'february') end up with similar vectors, because the same vectors must predict the same neighbours.

REAL-WORLD SCENARIO

Guessing what an unknown word means from the sentences it appears in: 'a glass of tesgüino' and 'tesgüino makes you drunk' already say it is a drink.

SYSTEM DIAGRAM



One (centre, context) pair, and two of the five noise words drawn for it.

Today's part for Day 90

`day86_exercise.py` builds the layer a language model needs instead of negative sampling: a full softmax over the vocabulary, forward and backward, gradient-checked. Day 90's GPT ends with it.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. From text to pairs

DEFINITION — start here, no prior knowledge assumed

Skip-gram never sees sentences as a whole. It sees (centre, context) pairs: every word paired with each word within a window around it. Word2Vec also shrinks the window at random for each centre (between 1 and 5 here), so near words are paired more often than far ones.

REAL-WORLD SCENARIO

A teacher who learns students' names from who sits next to whom, day after day.

IMPLEMENTATION

```
def skipgram_pairs(ids, window, rng):
    b = rng.randint(1, window + 1, size=len(ids))
    C, O = [], []
    for off in range(1, window + 1):
        right = b[:-off] >= off
        C.append(ids[:-off][right]); O.append(ids[off:][right])
        left = b[off:] >= off
        C.append(ids[off:][left]); O.append(ids[:-off][left])
    return np.concatenate(C), np.concatenate(O)
```

ACTUAL OUTPUT

```
text: anarchism originated as a term of abuse first
centre 'anarchism': pairs (anarchism, originated), (anarchism, as)
centre 'a': pairs (a, originated), (a, as), (a, term), (a, of)
Actual output from this lesson's run.
```

What it means

Each centre gives up to two pairs per side with a window of 2. Over 1.9 million tokens and a window of up to 5 that is millions of pairs per epoch; the script builds them with array slicing, one offset at a time, not word by word.

WHY CHOOSE THIS?

Simple, local and parallel: every pair is an independent example.

WHY NOT THIS? (TRADEOFFS)

Word order inside the window is ignored.

2. Two objectives: the full softmax and negative sampling

DEFINITION — start here, no prior knowledge assumed

The full version scores the context word against every vocabulary word and asks the softmax for its probability. Negative sampling asks a cheaper yes/no question: is this (centre, context) pair real, or one of k pairs whose context word was drawn at random from the vocabulary (the noise words)?

REAL-WORLD SCENARIO

Instead of picking the right answer out of 21,681 options, tell the real answer apart from five decoys.

MATHEMATICS

$$\text{full softmax: } loss = -\log \left[\exp(u_o \cdot v_c) / \Sigma_{w \text{ in } V} \exp(u_w \cdot v_c) \right]$$

$$\text{negative sampling: } loss = -\log s(u_o \cdot v_c) - \Sigma_{k \text{ noise words } n} \log s(-u_n \cdot v_c)$$

$$\text{noise words drawn with probability} \propto count^{0.75}; \quad s(z) = 1 / (1 + e^{-z})$$

IMPLEMENTATION

```
def sgns_loss_grads(Win, Wout, c, tgt):
    v, U = Win[c], Wout[tgt]
    z = np.einsum("nd,nkd->nk", v, U)
    loss = np.logaddexp(0, -z[:, 0]).sum() + np.logaddexp(0, z[:, 1:]).sum()
    gz = sigmoid(z); gz[:, 0] -= 1
    dv = np.einsum("nk,nkd->nd", gz, U)
    dU = gz[:, :, None] * v[:, None, :]
    return loss, dv, dU
```

ACTUAL OUTPUT

```
negative sampling  gradient check: 96 entries, largest |numeric - analytic| = 5.40e-10, all pass: True
full softmax       gradient check: 96 entries, largest |numeric - analytic| = 8.05e-10, all pass: True
one batch of 1024 pairs, V = 21681, d = 50: negative sampling (5 noise words) 1.8 ms, full softmax 519.8 ms --
    282 times slower (only the ratio is meaningful)
scores computed per pair: 6 against 21681
Actual output from this lesson's run.
```

What it means

Both losses and their gradients are right (5.40e-10 and 8.05e-10). The cost differs: negative sampling computes 6 scores per pair, the full softmax 21681, and one batch was 282 times slower with the full softmax in this run. At that speed the full softmax would need hours per epoch, so this lesson trains only with negative sampling.

WHY CHOOSE THIS?

Hundreds of times cheaper, and good word vectors.

WHY NOT THIS? (TRADEOFFS)

Not a probability over the vocabulary: a language model (Day 90) needs the full softmax.

3. Training, measured after every epoch

DEFINITION — start here, no prior knowledge assumed

Five passes over the corpus, three seeds. After every epoch, three checks of the vectors: **category** (of 90 words in 8 hand-made groups, how many have a nearest listed word from their own group), **capitals** (272 questions like paris:france :: rome:?) and **gender** (90 questions like king:queen :: man:?).

REAL-WORLD SCENARIO

Checking a student's progress after every chapter, not only in the final exam.

SPECIFICATION

Corpus	text8's first 2M words; words seen fewer than 5 times dropped: 21,681 words, 1.9M tokens.
Model	d = 50; window up to 5; 5 noise words; frequent words subsampled (t = 1e-4).
Training	5 epochs, batches of 1,024 pairs, learning rate 0.025 falling linearly to near 0.
Analogy rule	the answer is the word nearest to b – a + c, the three question words excluded.

Why this test

Categories test similarity; analogies test whether the differences between vectors mean something. Both are measured during training, so a late collapse or an early plateau would show.

IMPLEMENTATION

```
def train(words_ids, freq, V, seed, window=WINDOW, neg=NEG, t=SUBSAMPLE, epochs=EPOCHS, itos=None,
    stoi=None):
    rng = np.random.RandomState(seed + 100)
    model = SkipGram(V, seed=seed)
    cdf = noise_cdf(freq)
    total, done = epochs * len(words_ids), 0
    hist = []
    for ep in range(epochs):
        kept = subsample(words_ids, freq, t, rng) if t > 0 else words_ids
        C, O = skipgram_pairs(kept, window, rng)
        perm = rng.permutation(len(C)); C, O = C[perm], O[perm]
        loss = 0.0
        for s in range(0, len(C), BATCH):
            c, o = C[s:s + BATCH], O[s:s + BATCH]
            lr = LR0 * max(1e-4, 1 - (done + s / len(C) * len(words_ids)) / total)
            tgt = np.concatenate([o[:, None], np.searchsorted(cdf, rng.rand(len(c), neg))],
                axis=1)
            loss += model.step(c, tgt, lr)
        done += len(words_ids)
        hist.append((loss / len(C) / (1 + neg),) + evaluate(model.Win, itos, stoi))
    return model, np.array(hist)
```

ACTUAL OUTPUT

```
seed 0, after each epoch:
 epoch    loss  category  capitals  gender
     1   0.466     0.789     0.004   0.044
     2   0.413     0.878     0.029   0.089
     3   0.399     0.944     0.081   0.111
     4   0.391     0.944     0.147   0.200
     5   0.387     0.967     0.129   0.211
(loss: mean per scored candidate, the real pair or a noise word; 33 s per epoch here)
final scores per seed:
  seed  category  capitals  gender
     0     0.967     0.129   0.211
     1     0.967     0.125   0.267
     2     0.978     0.165   0.311
  mean     0.970     0.140   0.263
Actual output from this lesson's run.
```

What it means

The category check climbs from 0.789 after one epoch to 0.967, and the three seeds agree (mean 0.970). Analogies are far weaker: 0.140 for capitals and 0.263 for gender on average, and still rising at epoch 5. Similar words cluster with two million words; the regular offsets that analogies need want far more text (the original Word2Vec used billions of words).

WHY CHOOSE THIS?

Learns real structure from raw text alone, in minutes.

WHY NOT THIS? (TRADEOFFS)

Two million words are enough for clusters, not for reliable analogies.

4. What the vectors know

DEFINITION — start here, no prior knowledge assumed

The nearest words by cosine similarity, and six analogies answered with the top three candidates.

REAL-WORLD SCENARIO

Asking a well-read person 'what does this word remind you of?'

ACTUAL OUTPUT

```
  three     -> four, five, six, two, one, seven
  january   -> june, november, february, december, october, july
  france    -> spain, belgium, norway, italy, germany, netherlands
  red       -> cross, black, white, blue, coat, shorts
  physics   -> theoretical, mechanics, chemistry, relativity, mathematics, theory
  computer  -> computers, digital, portable, graphical, machines, gui
  war       -> outbreak, wars, liberated, invasion, revolutionary, battles
  music     -> dance, pop, musical, folk, difranco, musicians
  king      -> epirus, macedon, aragon, afonso, viii, portugal
  river     -> lake, rivers, situated, shore, inland, coast
analogies, a:b :: c:? -- the word nearest to b - a + c, question words excluded:
  paris:france :: rome:?  -> ruled, dominions, retained
  paris:france :: tokyo:?  -> overseas, japan, airways
  man:woman :: king:?  -> andronicus, throne, iv
  father:mother :: brother:?  -> husband, married, wife
  france:french :: germany:?  -> german, russian, austrian
  big:bigger :: small:?  -> absorbed, protected, rigid
Actual output from this lesson's run.
```

What it means

Numbers, months, countries and sciences find their own kind. 'king' finds Portuguese and Macedonian history: Wikipedia's kings appear in history articles. Of the six analogies only france:french :: germany:? is right first time ('german'); 'japan' is second for tokyo, and 'wife' third for brother:? (the right answer, 'sister', is missing). That matches the low analogy scores of Section 3.

WHY CHOOSE THIS?

Neighbours show what the corpus emphasises.

WHY NOT THIS? (TRADEOFFS)

Six examples are anecdotes; Section 3's scores are the measurement.

5. What each ingredient does

DEFINITION — start here, no prior knowledge assumed

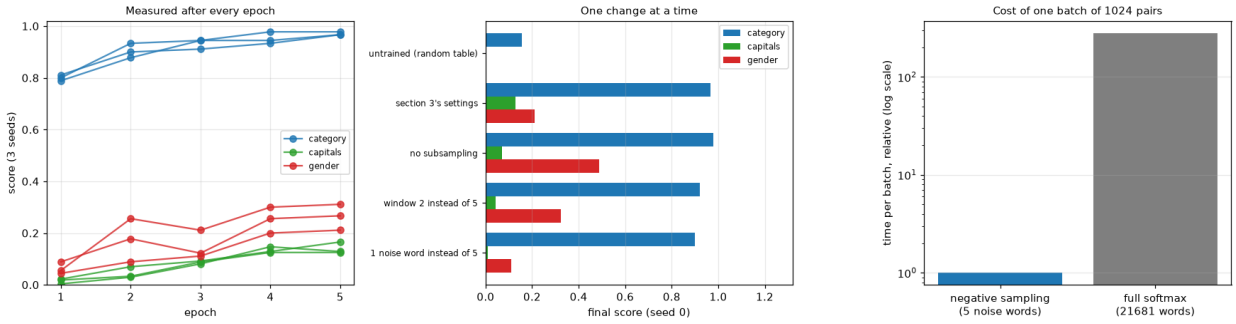
The same training with one setting changed at a time (seed 0), and the untrained starting table as the chance level.

REAL-WORLD SCENARIO

Baking the same cake four times, each time leaving out one ingredient.

ACTUAL OUTPUT

```
setting                       category  capitals  gender   s per epoch
untrained (random table)         0.156     0.000   0.000
section 3's settings             0.967     0.129   0.211
no subsampling                   0.978     0.070   0.489            62
window 2 instead of 5            0.922     0.044   0.322            16
1 noise word instead of 5        0.900     0.011   0.111            11
(seconds per epoch only for runs trained in this section; the section 3 run was timed there)
Actual output from this lesson's run.
```



Left: the three scores after every epoch, three seeds. Middle: one change at a time. Right: the cost of one batch.

What it means

Subsampling trades one analogy set for the other: without it capitals fall to 0.070 but gender rises to 0.489. A likely reason, not tested here: the gender questions use very frequent words ('he', 'she', 'man') that subsampling mostly throws away, while the capitals use rarer ones. One seed, so the size of the swing is uncertain too. A **window of 2** keeps categories (0.922) and loses capitals (0.044). **One noise word** is the cheapest and weakest on every score (0.900, 0.011, 0.111).

WHY CHOOSE THIS?

Shows that the settings matter, and in different ways for different words.

WHY NOT THIS? (TRADEOFFS)

Single seed per change; Section 3 showed seeds differ by a few points.

Interview preparation — technical questions

Q: What does skip-gram predict?

A: Each context word from the centre word alone.

Q: Why negative sampling?

A: The full softmax scores every vocabulary word per pair; here one batch was 282 times slower.

Q: Why draw noise words with count^0.75?

A: It flattens the distribution: rare words are drawn more often than their counts alone would give.

Q: What is subsampling?

A: Dropping each occurrence of a frequent word with a probability that grows with its frequency.

Q: How is an analogy answered?

A: The word nearest to b – a + c, the question words excluded (3CosAdd).

Interview preparation — theoretical questions

Q: Why do words with similar contexts get similar vectors?

A: The same vector has to predict the same neighbours, so the training pulls them together.

Q: Is skip-gram with negative sampling a language model?

A: No: it gives no probability over the vocabulary, only real-or-noise scores.

Q: Why do analogies need more data than categories?

A: They depend on consistent differences between vectors, which noise drowns at small sizes.

Q: Which vectors are kept, W_in or W_out?

A: Usually W_in, the centre-word vectors, as here; some methods add the two.

Key takeaway and what's next

KEY TAKEAWAY

Word2Vec's skip-gram learns useful word vectors from raw text by making each word predict its neighbours, and negative sampling makes that cheap. On two million words it gets word groups almost right but analogies mostly wrong, and its settings trade one kind of quality for another.

- Category check: 0.970 (three seeds) against 0.156 untrained.
- Analogies: capitals 0.140, gender 0.263 (mean of three seeds).
- Negative sampling: 6 scores per pair instead of 21,681; 282 times faster per batch here.
- Without subsampling: gender 0.489, capitals 0.070 (seed 0).

Suggested watch order, if starting from zero

- 1. Vectoring Words (Word Embeddings) (Computerphile) — what word vectors are, with examples.
- 2. Stanford CS224N, Lecture 1: Intro & Word Vectors (Stanford Online) — the skip-gram model and its training, from Stanford's NLP course.

Day 87 preview

- **Looking at the vectors.** PCA and t-SNE from scratch, the 50 numbers per word squeezed into a picture, and numbers that check how much of the structure the picture keeps.